

Reviewer Pack — Minimal Number of Automorphism Groups and Circuit Monotone W...

Entry f92a60664314 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 23:15:52 UTC

Minimal Number of Automorphism Groups and Circuit Monotone Width Correlation

Entry ID: f92a60664314 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 23:15:52 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (specifically, automorphism groups of graphs) **Field B** (complexity object): Boolean Circuits

Statement:

For every n -vertex graph G with maximum degree $\Delta(G)$, the minimal number of distinct automorphism groups of subgraphs of G is linearly correlated with the circuit monotone width $w_m(G)$ of G , such that the ratio of the number of automorphism groups to the square root of the circuit monotone width is bounded by a constant factor, i.e., $|\text{Aut}(G)| / \sqrt{w_m(G)} \leq c$ for some constant c .

Rationale (proposer's reasoning):

Automorphism groups capture symmetries in graphs and could potentially reveal underlying structures that influence computational complexity. Circuit monotone width measures the size of the smallest monotone circuit computing a given function, which is related to the complexity of problems like satisfiability. A correlation between these two would suggest a deep connection between algebraic properties of graphs and their computational complexity.

Taxonomy category: GROUP_THEORY_AUTOMORPHISM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9612137f0d917dd9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every n -vertex graph G with maximum degree $\Delta(G) \leq 40$, the ratio of the number of automorphism groups to the square root of the circuit monotone width is within an acceptable constant factor c , where 'acceptable' is defined as $|\text{Aut}(G)| / \sqrt{w_m(G)} \leq c$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - aut(Aut(G)), graph automorphism groups AND Boolean circuits - circuit monotone width $w_m(G)$ AND group theory automorphisms - linear correlation between number of automorphism groups and circuit monotone width for graphs

Top relevant hits considered: - [s2:10.2989/16073606.2025.2544235] On automorphism groups of the conjugacy class type Cayley graphs on the symmetric and alternating groups - [s2:10.1017/s0008439500029453] Lectures on Boolean algebras, by P.R. Halmos. Van Nostrand. iv + 147 pages. \$2.95.

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def generate_random_graph(n, delta):
    G = defaultdict(set)
    for _ in range(delta * n // 2):
        u, v = random.sample(range(n), 2)
        if u != v and u not in G[v]:
            G[u].add(v)
            G[v].add(u)
    return G

def compute_automorphism_groups(G):
    def dfs(node, path, visited):
        if node in visited:
            return
        visited.add(node)
        path.append(node)
        for neighbor in G[node]:
            dfs(neighbor, path, visited)

    def is_isomorphic(G1, G2):
        if len(G1) != len(G2):
            return False
```

```

nodes = list(G1.keys())
random.shuffle(nodes)
mapping = {nodes[0]: next(iter(G2))}
stack = [(nodes[0], next(iter(G2)))]
visited = set([nodes[0]])

while stack:
    u, v = stack.pop()
    for neighbor in G1[u]:
        if neighbor not in visited:
            visited.add(neighbor)
            w = next((w for w in G2[v] if len(set(G1[neighbor]) - set(G2[w])) == 0), None)
            if w is None or w in mapping.values():
                return False
            mapping[neighbor] = w
            stack.append((neighbor, w))

    return True

automorphism_groups = []
for perm in itertools.permutations(range(len(G))):
    G_perm = {perm[i]: set(perm[j] for j in G[i]) for i in range(len(G))}
    if is_isomorphic(G, G_perm):
        automorphism_groups.append(frozenset(perm))

return len(set(automorphism_groups))

def compute_circuit_monotone_width(G):
    n = len(G)
    w_m = float('inf')

    def dfs(i, path):
        nonlocal w_m
        if i in path:
            cycle_length = len(path) - path.index(i)
            w_m = min(w_m, cycle_length)
            return
        path.append(i)
        for neighbor in G[i]:
            dfs(neighbor, path)
        path.pop()

    for start in range(n):
        dfs(start, [])

    return w_m

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 30
    delta = 40

    G = generate_random_graph(n, delta)
    aut_groups = compute_automorphism_groups(G)
    w_m = compute_circuit_monotone_width(G)

```

```

metric_value = aut_groups / math.sqrt(w_m) if w_m != 0 else float('inf')
conjecture_holds = metric_value <= 1.5 # Placeholder constant for demonstration
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Aut(G) / sqrt(w_m)",
    "metric_value": metric_value,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13587
2	propose	ollama_remote	glm4:latest	0	0	14513
3	propose	ollama_remote	glm4:latest	0	0	13654
4	preregistration	ollama_remote	glm4:latest	0	0	10538
5	novelty	ollama_remote	glm4:latest	0	0	11918
6	novelty	ollama_remote	glm4:latest	0	0	17966
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18583
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11952
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15251
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16590
11	judge	ollama_remote	glm4:latest	0	0	8452

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 153004 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/f92a60664314.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f92a60664314.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f92a60664314.tar.gz (if generated)